



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

DOCUMENTAZIONE PER PROGETTO BASI DI DATI

CdL Triennale in Informatica

Gabriella Scaraglia [N86005338], Bartolomeo Russo [N86005210],

Matteo Oliva [N86004167]

CORSO DI BASI DI DATI, 2024/2025

Indice

1	Introduzione	4
1.1	Descrizione del problema	4
2	Progettazione Concettuale	4
2.1	Class Diagram	4
2.2	Ristrutturazione del Class Diagram	5
2.2.1	Analisi delle chiavi	5
2.2.2	Analisi degli attributi secondari	5
2.2.3	Analisi delle ridondanze	6
2.2.4	Analisi degli attributi strutturati	6
2.2.5	Analisi degli attributi a valore multiplo	6
2.2.6	Analisi delle gerarchie di specializzazione	7
2.3	Class Diagram Ristrutturato	7
2.4	Dizionario delle Classi	8
2.5	Dizionario delle Associazioni	11
2.6	Dizionario dei Vincoli	12
3	Progettazione Logica	13
3.1	Schema Logico	13
4	Progettazione Fisica	15
4.1	Definizione Tabelle	15
4.1.1	Definizione della tabella Utente	15
4.1.2	Definizione della tabella Bacheca	16
4.1.3	Definizione della tabella Task	17
4.1.4	Definizione della tabella Link	18
4.1.5	Definizione della tabella Allegato	19
4.1.6	Definizione della tabella ChecklistItem	19
4.1.7	Definizione della tabella Condivisione	20
4.2	Implementazione dei Vincoli	21
4.2.1	Gestione permesso Proprietario	21
4.2.2	Gestione auto-condivisione	22
4.2.3	Gestione limite bacheche	23
4.3	Funzioni, Procedure e altre automazioni	24
4.3.1	Gestione creazione tabelle	24
4.3.2	Gestione completamento task	25
4.3.3	Ricerca task per deadline	26
4.3.4	Ricerca task per titolo	27

URL a GITHUB

<https://github.com/BartolomeoRusso9/ProgettoBD-GestoreToDo>

1 Introduzione

1.1 Descrizione del problema

Il sistema informativo ToDo è composto da un database relazionale (PostgreSQL) che permette all'utente, una volta registratosi, di creare e gestire Task ("ToDo") all'interno di tre Bacheche, modificabili dall'utente: Università, Lavoro e Tempo Libero.

Le Task hanno come elementi: titolo, descrizione, data di creazione, data di scadenza, link, colore di sfondo e allegato.

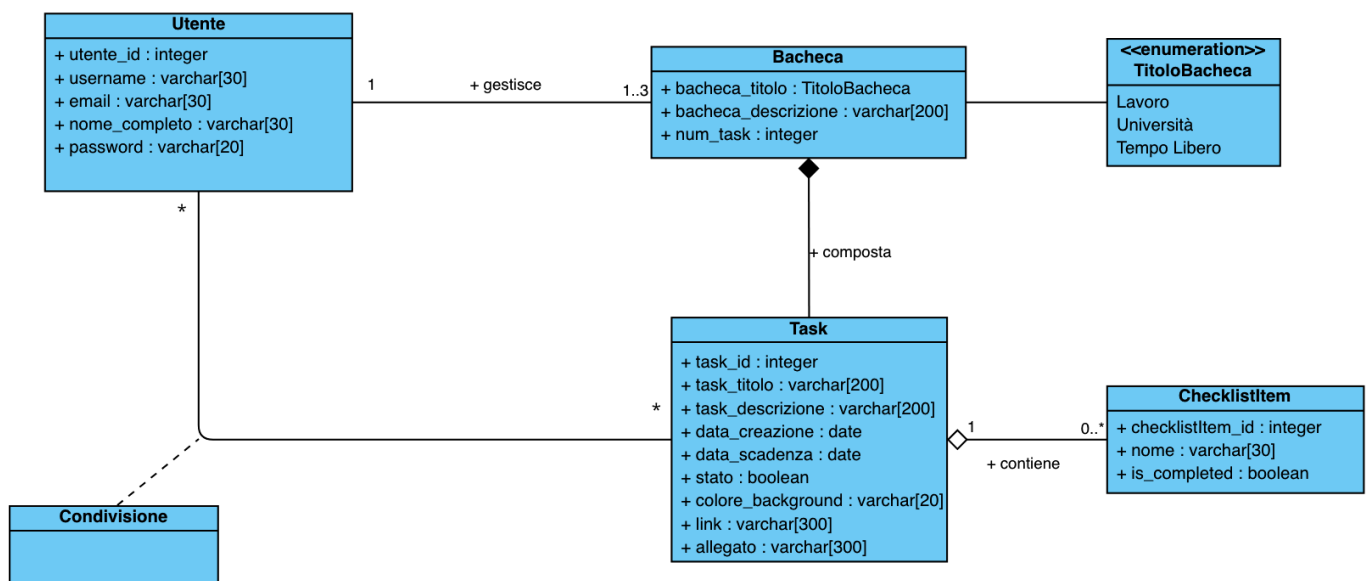
All'interno di ogni Task è possibile inserire una CheckList di sotto-attività.

Inoltre, è possibile convalidare qualsiasi Task con altri utenti registrati nel sistema.

La Task condivisa può essere gestita unicamente dall'autore.

2 Progettazione Concettuale

2.1 Class Diagram



2.2 Ristrutturazione del Class Diagram

Si procede alla ristrutturazione del Class Diagram iniziale per eliminare eventuali inefficienze e renderlo compatibile con il modello relazionale, seguendo i passaggi standard di progettazione.

2.2.1 Analisi delle chiavi

Per ogni entità definita nel sistema, si è scelto di adottare chiavi primarie di tipo **surrogato**. Nello specifico, le entità dispongono di un attributo ID di tipo intero, autoincrementale e privo di significato semantico nel dominio applicativo.

A differenza delle chiavi naturali, l'uso di ID ottimizza drasticamente l'efficienza del sistema.

Aggiunto nell'entità *Bacheca*.

2.2.2 Analisi degli attributi secondari

In questa fase si è proceduto all'individuazione di eventuali attributi derivati, i cui valori sono definiti a partire da altri attributi.

Non vengono memorizzati in modo persistente all'interno del database, bensì calcolati dinamicamente a runtime.

A seguito dell'analisi, si è stabilito di non procedere alla rimozione di tali entità informative, poiché non sono stati riscontrati attributi il cui calcolo a runtime risulti oneroso al punto da compromettere l'efficienza o da giustificare una modifica.

2.2.3 Analisi delle ridondanze

Una ridondanza corrisponde alla presenza di un dato che può essere derivato da altri dati.

Nel modello iniziale, l'attributo `num_task` nell'entità *Bacheca* è stato identificato come ridondante, poiché il suo valore è ottenibile a runtime tramite la relazione di associazione che lega ogni bacheca alle rispettive task. Si è pertanto proceduto alla sua rimozione per garantire la coerenza dei dati del modello.

2.2.4 Analisi degli attributi strutturati

Ciò si verifica quando un attributo è composto da più sottocampi.

Nel diagramma originale, gli attributi `link` e `allegato` all'interno di *Task* sono stati identificati come tali.

In fase di ristrutturazione, si è scelto di scomporre questi elementi migrandoli verso le entità dedicate *Link* e *Allegato*.

2.2.5 Analisi degli attributi a valore multiplo

Un attributo è multivalore quando un'entità può avere più valori per lo stesso campo.

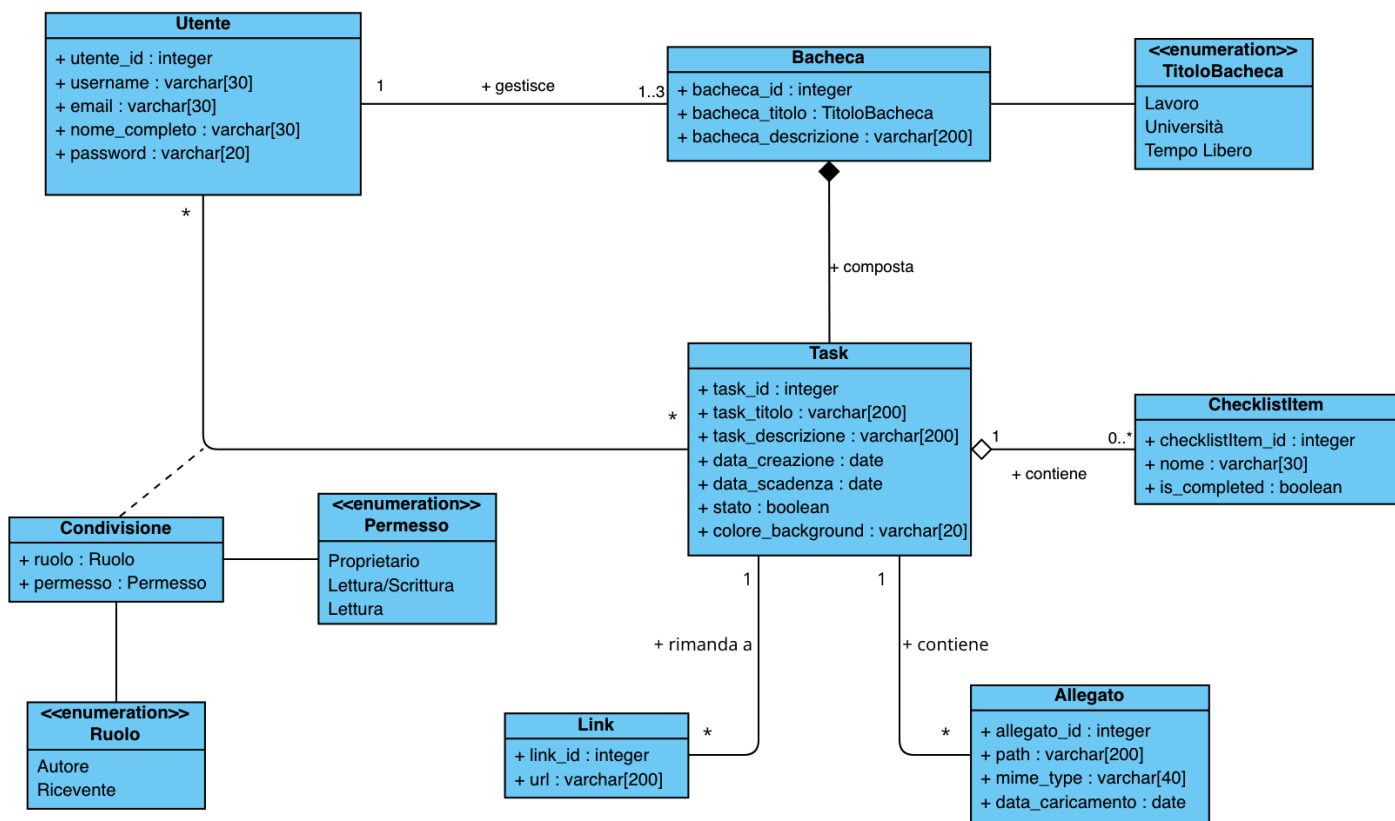
Per risolvere tale problematica e garantire la scalabilità del sistema, si è proceduto alla creazione di nuove entità indipendenti, collegate a *Task* tramite relazioni di associazione: *Link*, per gestire i riferimenti esterni (`url`), e *Allegato*, per gestire le risorse multimediali con la separazione della logica del file (`path`, `mimeType`, `data_caricamento`).

2.2.6 Analisi delle gerarchie di specializzazione

In questa fase della progettazione, si è proceduto alla verifica della presenza di eventuali gerarchie di specializzazione tra le entità del modello.

La struttura delle entità è stata mantenuta, in quanto non sono state riscontrate specializzazioni o generalizzazioni tali da giustificare una ristrutturazione.

2.3 Class Diagram Ristrutturato



2.4 Dizionario delle Classi

Classe	Descrizione	Attributi
Utente	Modella il profilo di chi usufruisce dei servizi, garantendo l'identificazione tramite credenziali univoche.	• utente_id (<i>integer</i>): Identificativo univoco. (PK) • username (<i>varchar[30]</i>): Nome dell'account. • email (<i>varchar[30]</i>): Indirizzo di posta elettronica. • nome_completo (<i>varchar[30]</i>): Nome e cognome dell'utente. • password (<i>varchar[20]</i>): Chiave di autenticazione.
Bacheca	Contenitore logico di task, categorizzato in base all'ambito (Università, Lavoro, Tempo Libero).	• bacheca_id (<i>integer</i>): Identificativo univoco. (PK) • bacheca_titolo (<i>TitoloBacheca</i>): Titolo definito tramite enumerazione. • bacheca_descrizione (<i>varchar[200]</i>): Descrizione delle finalità della bacheca.

Classe	Descrizione	Attributi
Task	Singola attività inserita dall'utente all'interno di una bacheca.	• <u>task_id</u> (<i>integer</i>): Identificativo univoco. (PK) • task_titolo (<i>varchar[200]</i>): Titolo dell'attività. • task_descrizione (<i>varchar[200]</i>): Dettagli della task. • data_creazione (<i>date</i>): Data creazione in formato ISO 8601 (YYYY-MM-DD). • data_scadenza (<i>date</i>): Data scadenza in formato ISO 8601 (YYYY-MM-DD). • stato (<i>boolean</i>): Flag di completamento (true/false). • colore_background (<i>varchar[20]</i>): Codice HEX del colore per la personalizzazione dello sfondo.
Link	Riferimento ipertestuale esterno collegato a una task.	• <u>link_id</u> (<i>integer</i>): Identificativo univoco. (PK) • url (<i>varchar[200]</i>): Indirizzo web della risorsa.

Classe	Descrizione	Attributi
Allegato	File fisico collegato opzionalmente a una task.	• <u>allegato_id</u> (<i>integer</i>): Identificativo univoco. (PK) • path (<i>varchar[200]</i>): Percorso nel file system. • mime_type (<i>varchar[40]</i>): Formato del file. • data_caricamento (<i>date</i>): Data di caricamento dell'allegato.
Checklist Item	Sotto-attività atomica interna a una task per monitorarne l'avanzamento.	• <u>checklistItem_id</u> (<i>integer</i>): Identificativo univoco. (PK) • nome (<i>varchar[30]</i>): Descrizione breve della sotto-task. • is_completed (<i>boolean</i>): Stato di completamento della voce.
Condivisione	Classe associativa che gestisce la condivisione di task tra utenti.	• ruolo (<i>Ruolo</i>): Specifica se l'utente è Autore o Ricevente tramite enumerazione. • permesso (<i>Permesso</i>): Livello di accesso (Proprietario, Lettura/Scrittura, Lettura).

2.5 Dizionario delle Associazioni

Nome	Descrizione	Classi coinvolte
GESTISCE	Indica che ogni utente può possedere delle bacheche personali, con il limite di una per ogni tipologia esistente.	● Utente 1 ● Bacheca 1..3 ● Note: Ogni utente può avere da 1 a 3 bacheche e ogni bacheca appartiene a un solo utente.
COMPOSTA DA	Indica il legame di composizione tra la bacheca e le attività, dove l'esistenza delle task dipende dalla bacheca.	● Bacheca 1 ● Task * ● Note: La cancellazione della bacheca comporta la rimozione automatica di tutte le task associate.
RIMANDA A	Può associare ad ogni singola attività uno o più riferimenti ipertestuali esterni.	● Task 1 ● Link * ● Note: Un link non può esistere senza una task di riferimento.
CONTIENE	Può associare ad ogni singola attività uno o più allegati.	● Task 1 ● Allegato * ● Note: Un allegato non può esistere senza una task di riferimento.
CONTIENE	Può associare ad ogni singola attività una o più sotto-attività.	● Task 1 ● ChecklistItem * ● Note: Una sotto-attività non può esistere senza una task di riferimento.
CONDIVIDE	Descrive la relazione multi-a-molti tra utenti e task, permettendo la collaborazione con diversi livelli di permesso.	● Utente * ● Task * ● Note: Ogni task condivisa deve avere un unico utente con ruolo di Proprietario.

2.6 Dizionario dei Vincoli

Vincolo	Descrizione
credenziali_univoche	Ogni username deve essere univoco all'interno del sistema per identificare il soggetto.
credenziali_univoche	Non è consentita la registrazione di più account con lo stesso indirizzo email .
pk_condivisione	Non deve esistere più di una riga in <i>Condivisione</i> per la stessa combinazione di utente e task.
trigger_proprietario_unico	Per ogni task condivisa, deve esistere un solo utente con permesso Proprietario .
trigger_previeni_autocondivisione	L'utente non può condividere la task con sè stesso.
check_email_format	L'attributo deve rispettare il formato standard <i>indirizzo@dominio.estensione</i> .
check_minLength_password	La stringa deve avere una lunghezza minima di 8 caratteri per garantire la sicurezza.
check_url_format	Il campo url della classe <i>Link</i> deve essere un indirizzo web sintatticamente valido (protocollo HTTP/HTTPS).
check_deadline_valida	La data di scadenza deve essere uguale o successiva alla data di creazione della task.
check_data_caricamento_valida	La data di caricamento di un allegato non può essere nel futuro.
Dominio Bacheca	I valori ammissibili sono limitati a quelli definiti nell'enumeration <i>TitoloBacheca</i> .
Dominio Ruolo	I valori ammissibili sono limitati a quelli definiti nell'enumeration <i>Ruolo</i> .
Dominio Permesso	I valori ammissibili sono limitati a quelli definiti nell'enumeration <i>Permesso</i> .

Vincolo	Descrizione
trigger_limite_bacheche	Ciascun <i>utente</i> può possedere al massimo una <i>bacheca</i> per ogni tipologia definita (<i>TitoloBacheca</i>).

3 Progettazione Logica

In questa fase traduciamo le strutture concettuali in strutture logiche gestite dal DBMS.

3.1 Schema Logico

Le chiavi primarie (PK) sono indicate con sottolineatura e **grassetto**.

Le chiavi esterne (FK) sono indicate con sottolineatura e *corsivo*.

- **Utente** (**utente_id**, username, email, nome_completo, password)
- **Bacheca** (**bacheca_id**, bacheca_titolo, bacheca_descrizione, *utente_id*)
 - *utente_id* → Utente.utente_id
- **Task** (**task_id**, task_titolo, task_descrizione, data_creazione, data_scadenza, stato, colore_background, *bacheca_id*)
 - *bacheca_id* → Bacheca.bacheca_id
- **Link** (**link_id**, url, *task_id*)
 - *task_id* → Task.task_id

- **Allegato** (allegato_id, path, mime_type, data_caricamento, task_id)
 - task_id $\rightarrow$ Task.task_id
- **ChecklistItem** (checklistItem_id, nome, is_completed, task_id)
 - task_id $\rightarrow$ Task.task_id
- **Condivisione** (ruolo, permesso, utente_id, task_id)
 - utente_id $\rightarrow$ Utente.utente_id
 - task_id $\rightarrow$ Task.task_id

4 Progettazione Fisica

Traduciamo lo schema logico nella struttura fisica effettiva che verrà implementata in PostgreSQL.

In questa fase, si considerano le caratteristiche hardware e software del sistema, ottimizzando le prestazioni e l'accesso ai dati.

4.1 Definizione Tabelle

4.1.1 Definizione della tabella Utente

```
CREATE TABLE Utente(  
  utente_id      SERIAL          PRIMARY KEY,  
  username       varchar(30)     NOT NULL,  
  email          varchar(30)     NOT NULL,  
  nome_completo  varchar(30),  
  password       varchar(20)     NOT NULL,  
  
  CONSTRAINT credenziali_univoche UNIQUE(username, email),  
  
  CONSTRAINT check_email_format  
  CHECK (email ~ '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.(it|com)$'),  
  
  CONSTRAINT check_minLength_password CHECK (LENGTH(password) ≥ 8)  
);
```

4.1.2 Definizione della tabella Bacheca

```
CREATE TYPE TitoloBacheca AS ENUM ('Università', 'Lavoro', 'Tempo Libero');

CREATE TABLE Bacheca(
  bacheca_id          SERIAL          PRIMARY KEY,
  bacheca_titolo       TitoloBacheca NOT NULL,
  bacheca_descrizione  varchar(200),
  utente_id           integer         NOT NULL,

  CONSTRAINT fk_utente FOREIGN KEY (utente_id)
  REFERENCES Utente(utente_id)
  ON DELETE CASCADE
);
```


4.1.3 Definizione della tabella Task

```
CREATE TABLE Task(  
  task_id          SERIAL          PRIMARY KEY,  
  task_titolo      varchar(200),  
  task_descrizione varchar(200),  
  data_creazione   date            NOT NULL DEFAULT CURRENT_DATE,  
  data_scadenza    date,  
  stato            boolean          NOT NULL DEFAULT FALSE,  
  colore_background varchar(20)     NOT NULL DEFAULT '#FFFFFF',  
  bacheca_id       integer          NOT NULL,  
  
  CONSTRAINT fk_bacheca FOREIGN KEY (bacheca_id)  
  REFERENCES Bacheca(bacheca_id)  
  ON UPDATE CASCADE  
  ON DELETE CASCADE,  
  
  CONSTRAINT check_deadline_valida_  
  CHECK (data_scadenza ≥ data_creazione),  
  
  CONSTRAINT check_hex_format  
  CHECK (colore_background ~ '^#[0-9a-fA-F]{6}$')  
);
```

4.1.4 Definizione della tabella Link

```
CREATE TABLE Link(  
  link_id    SERIAL          PRIMARY KEY,  
  url        varchar(200),  
  task_id    integer         NOT NULL,  
  
  CONSTRAINT fk_task FOREIGN KEY (task_id)  
  REFERENCES Task(task_id)  
  ON DELETE CASCADE,  
  
  CONSTRAINT check_url_format  
  CHECK (url ~* '^https?://[a-zA-Z0-9\-\.\.]+\.[a-zA-Z]{2,}(/.*)?$')  
);
```

4.1.5 Definizione della tabella Allegato

```
CREATE TABLE Allegato(  
  allegato_id      SERIAL          PRIMARY KEY,  
  path             varchar(200),  
  mime_type        varchar(40),  
  data_caricamento date,  
  task_id          integer         NOT NULL,  
  
  CONSTRAINT fk_task FOREIGN KEY (task_id)  
  REFERENCES Task(task_id)  
  ON DELETE CASCADE,  
  
  CONSTRAINT check_data_caricamento_valida  
  CHECK (CURRENT_DATE ≥ data_caricamento)  
);
```

4.1.6 Definizione della tabella ChecklistItem

```
CREATE TABLE ChecklistItem(  
  checklistItem_id SERIAL          PRIMARY KEY,  
  nome             varchar(30),  
  is_completed      boolean        NOT NULL DEFAULT FALSE,  
  task_id          integer         NOT NULL,  
  
  CONSTRAINT fk_task FOREIGN KEY (task_id)  
  REFERENCES Task(task_id)  
  ON DELETE CASCADE  
);
```

4.1.7 Definizione della tabella Condivisione

```
CREATE TYPE Ruolo AS ENUM ('Autore', 'Ricevente');
CREATE TYPE Permessi AS ENUM ('Proprietario', 'Lettura/Scrittura', 'Lettura');

CREATE TABLE Condivisione(
  ruolo      Ruolo      NOT NULL,
  permesso   Permessi   NOT NULL,
  utente_id  integer     NOT NULL,
  task_id    integer     NOT NULL,

  CONSTRAINT pk_condivisione PRIMARY KEY (utente_id, task_id),

  CONSTRAINT fk_utente FOREIGN KEY (utente_id)
    REFERENCES Utente(utente_id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,

  CONSTRAINT fk_task FOREIGN KEY (task_id)
    REFERENCES Task(task_id)
    ON UPDATE CASCADE
    ON DELETE CASCADE
);
```

4.2 Implementazione dei Vincoli

4.2.1 Gestione permesso Proprietario

Ogni task condivisa può avere massimo un Proprietario.

```
CREATE OR REPLACE FUNCTION check_proprietario_unico()
RETURNS TRIGGER AS $$
BEGIN
    IF (NEW.permesso = 'Proprietario' AND EXISTS (
        SELECT 1
        FROM Condivisione
        WHERE task_id = NEW.task_id
              AND permesso = 'Proprietario'
              AND utente_id ≠ NEW.utente_id
    )) THEN
        RAISE EXCEPTION 'ERRORE: La task di ID % ha già un proprietario assegnato!', NEW.task_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_proprietario_unico
BEFORE INSERT OR UPDATE ON Condivisione
FOR EACH ROW
EXECUTE FUNCTION check_proprietario_unico();
```

4.2.2 Gestione auto-condivisione

Un utente non può condividere una task con sè stesso.

```
CREATE OR REPLACE FUNCTION previeni_autocondivisione()
    RETURNS TRIGGER AS $$
DECLARE
    autore_id INTEGER;
BEGIN
    SELECT b.utente_id INTO autore_id
    FROM Task t
        JOIN Bacheca b ON t.bacheca_id = b.bacheca_id
    WHERE t.task_id = NEW.task_id;

    IF NEW.utente_id = autore_id THEN
        RAISE EXCEPTION 'ERRORE: Non è possibile condividere un task con il suo stesso proprietario!';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_previeni_autocondivisione
    BEFORE INSERT OR UPDATE ON Condivisione
    FOR EACH ROW
EXECUTE FUNCTION previeni_autocondivisione();
```

4.2.3 Gestione limite bacheche

Ogni utente ha il limite di una bacheca per ambito (Università, Lavoro, Tempo Libero).

```
CREATE OR REPLACE FUNCTION check_limite_bacheche()
RETURNS TRIGGER AS $$
DECLARE
    conteggio_tipo INTEGER;
BEGIN
    SELECT COUNT(*)
    INTO conteggio_tipo
    FROM Bacheca
    WHERE utente_id = NEW.utente_id
        AND bacheca_titolo = NEW.bacheca_titolo;

    IF conteggio_tipo ≥ 1 THEN
        RAISE EXCEPTION 'ERRORE: L''utente di ID % possiede già una bacheca di tipo "%"! Massimo
consentito: 1 per tipo di Bacheca.',
            NEW.utente_id, NEW.bacheca_titolo;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_limite_bacheche
BEFORE INSERT ON Bacheca
FOR EACH ROW
EXECUTE FUNCTION check_limite_bacheche();
```

4.3 Funzioni, Procedure e altre automazioni

4.3.1 Gestione creazione tabelle

Permette la creazione automatica delle 3 bacheche di default (Università, Lavoro, Tempo Libero) dopo l'inserimento di un nuovo utente nella tabella Utente, tramite un trigger AFTER INSERT.

```
CREATE OR REPLACE FUNCTION crea_bacheche_default()
    RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Bacheca (bacheca_titolo, bacheca_descrizione, utente_id)
    VALUES
        ('Università', NULL, NEW.utente_id),
        ('Lavoro', NULL, NEW.utente_id),
        ('Tempo Libero', NULL, NEW.utente_id);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_crea_bacheche
    AFTER INSERT ON Utente
    FOR EACH ROW
EXECUTE FUNCTION crea_bacheche_default();
```


4.3.2 Gestione completamento task

La funzione aggiorna automaticamente lo stato di una task in base al completamento della sua checklist.

Viene eseguita dopo un'operazione di **INSERT** o di **UPDATE** del campo *is_completed* nella tabella *ChecklistItem*.

La funzione conta il numero totale di elementi della checklist associati al task e quanti di questi risultano completati.

Se tutti gli elementi sono spuntati, imposta il campo stato della task a **TRUE**; in caso contrario (incluso quando una spunta viene rimossa), lo imposta a **FALSE**. In questo modo, lo stato della task riflette sempre automaticamente il progresso della checklist.

```
CREATE OR REPLACE FUNCTION controlla_completamento_task()
    RETURNS TRIGGER AS $$
DECLARE
    totale_item INTEGER;
    item_completati INTEGER;
BEGIN
    SELECT COUNT(*) INTO totale_item FROM ChecklistItem WHERE task_id = NEW.task_id;

    SELECT COUNT(*) INTO item_completati
    FROM ChecklistItem
    WHERE task_id = NEW.task_id AND is_completed = TRUE;

    IF totale_item > 0 AND totale_item = item_completati THEN
        UPDATE Task SET stato = TRUE WHERE task_id = NEW.task_id;
    ELSE
        UPDATE Task SET stato = FALSE WHERE task_id = NEW.task_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_checklist_completamento
    AFTER INSERT OR UPDATE OF is_completed ON ChecklistItem
    FOR EACH ROW
EXECUTE FUNCTION controlla_completamento_task();
```

4.3.3 Ricerca task per deadline

La funzione restituisce l'elenco delle task di un determinato utente con scadenza entro una data specificata.

Accetta come parametro obbligatorio l'ID dell'utente e un parametro opzionale che rappresenta la data limite, impostata di default alla data corrente.

La funzione seleziona le task associate alle bacheche dell'utente che hanno una data di scadenza non nulla e precedente o uguale alla data indicata.

Il risultato include ID, titolo, data di scadenza, titolo della bacheca e stato della task, ordinati per data di scadenza crescente.

```
CREATE OR REPLACE FUNCTION task_in_scadenza(  
    p_utente_id INTEGER,  
    p_fino_a    DATE DEFAULT CURRENT_DATE  
)  
  
    RETURNS TABLE (  
        task_id    INTEGER,  
        task_titolo VARCHAR(200),  
        data_scadenza DATE,  
        bacheca_titolo TitoloBacheca,  
        stato      BOOLEAN  
    ) AS $$  
  
BEGIN  
    RETURN QUERY  
        SELECT  
            t.task_id,  
            t.task_titolo,  
            t.data_scadenza,  
            b.bacheca_titolo,  
            t.stato  
        FROM Task t  
            JOIN Bacheca b ON t.bacheca_id = b.bacheca_id  
        WHERE b.utente_id = p_utente_id  
            and t.data_scadenza ≤ p_fino_a  
            and t.data_scadenza IS NOT NULL  
        ORDER BY t.data_scadenza;  
  
END;  
$$ LANGUAGE plpgsql;
```

4.3.4 Ricerca task per titolo

La funzione consente di cercare le task di un determinato utente filtrandole in base a una parola chiave presente nel titolo.

Accetta l'ID dell'utente e una stringa di ricerca, restituendo tutte le task appartenenti alle sue bacheche il cui titolo contiene la parola specificata.

La ricerca è case-insensitive grazie all'utilizzo di **ILIKE**, permettendo anche corrispondenze parziali.

Il risultato include ID, titolo, data di scadenza, bacheca di appartenenza e stato del task, ordinati per titolo della bacheca e titolo della task.

```
CREATE OR REPLACE FUNCTION cerca_task_per_titolo(
  p_utente_id INTEGER,
  p_keyword   VARCHAR
)
RETURNS TABLE (
  task_id      INTEGER,
  task_titolo  VARCHAR(200),
  data_scadenza DATE,
  bacheca_titolo TitoloBacheca,
  stato        BOOLEAN
) AS $$
BEGIN
  RETURN QUERY
    SELECT
      t.task_id,
      t.task_titolo,
      t.data_scadenza,
      b.bacheca_titolo,
      t.stato
    FROM Task t
      join Bacheca b ON t.bacheca_id = b.bacheca_id
   WHERE b.utente_id = p_utente_id
      and t.task_titolo ILIKE '%' || p_keyword || '%'
   ORDER BY b.bacheca_titolo, t.task_titolo;
END;
$$ LANGUAGE plpgsql;
```